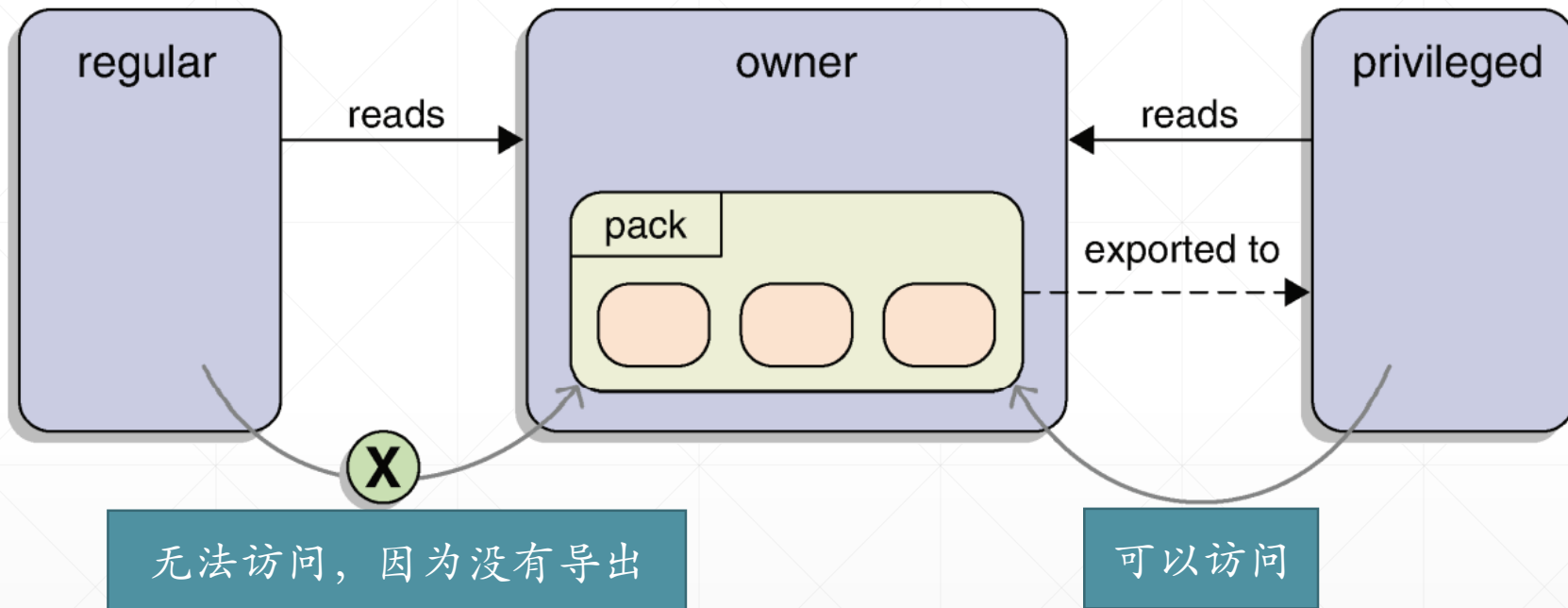


限定导出与开放反射

北京理工大学计算机学院
金旭亮



限定导出：对模块的可读性进行微调



owner模块中的pack包，对左侧的regular模块“封闭”，对右侧的privileged模块“开放”，要实现这种“有限制”的“开放”，就需要使用JPMS中的“限定导出”特性。

限定导出 (exports ... to ...) 语法

```
module my_module{  
    exports mypackage to other_module;  
}
```

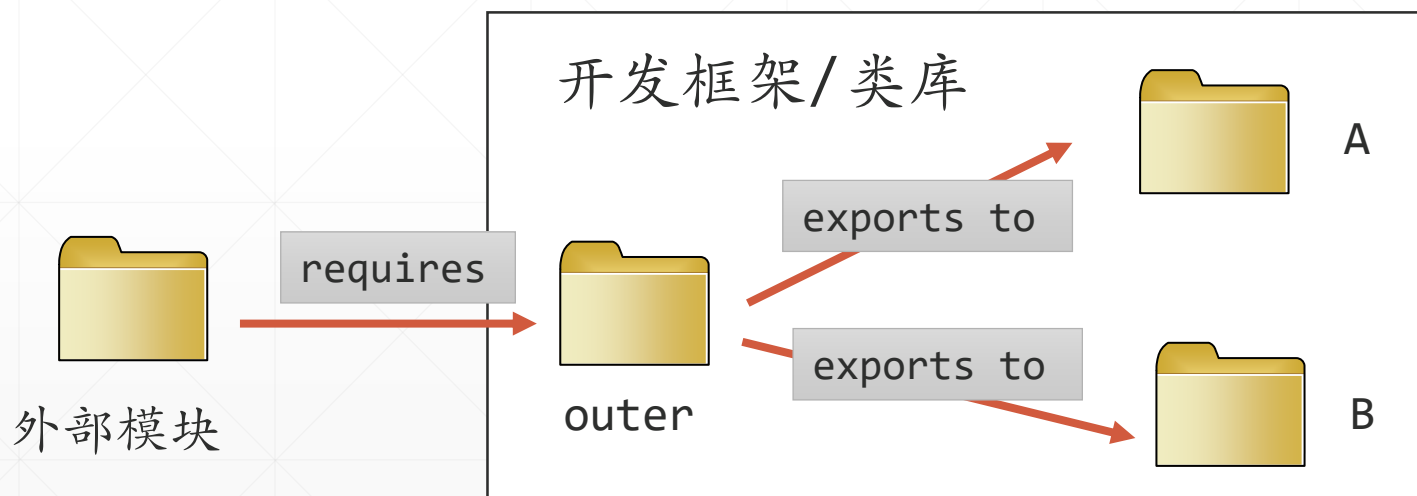
这么定义了之后，只有other_module模块可以访问my_module模块中的mypackage包。



应该避免在应用程序的模块之间大量使用限制导出。因为使用限制导出意味着在导出模块和允许的使用者之间建立了直接的联系，如果这种情况出现太多，意味着代码之间的耦合性过强，你的设计可能需要调整。

限定导出的应用场景

限定导出使多个模块可以共享同一个包，又不至于使该包对同一个JVM中的所有其他模块可见。这使得限定导出对于包含多个模块的类库和框架尤为有用，因为它可以用于在它们之间共享代码，同时可以避免客户端使用这些代码。对于那些想限制外界访问特定API的应用程序来说，它也非常有用。



如果需要的话，可以在java命令行中使用`--add-exports`，动态地将某个包开放给其他模块。

反射

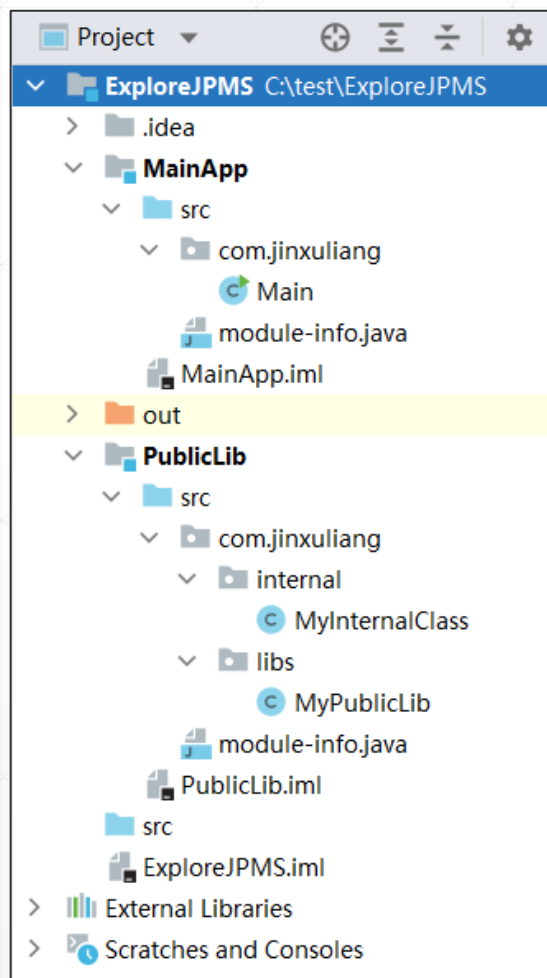


反射API允许框架访问非公有类和非公有成员。在许多框架（比如Hibernate，Spring）中，反射都有着重要的应用，它们的强大功能与灵活特性，很多都依赖于应用反射技术。



但问题是，在模块化系统中反射不再是不受限制的了。在JPMS的约束之下，外部模块只能访问可读模块中显式导出包中公有类型的公有成员，反射访问非导出的类型，反射存取特定类型中非公有的成员，都是被禁止的。为了让这些框架可以继续使用，JPMS必须对这些早期编写的反射代码进行特殊的处理，以便让它们可以继续工作。

示例分析



示例程序中包容有两个模块，MainApp模块读取PublicLib模块。

```
module-info.java (MainApp) x
1  module MainApp {
2      requires PublicLib;
3  }
```

PublicLib模块里有两个包，其中，com.jinxuliang.libs这个包是导出的，而com.jinxuliang.internal这个包是“私有”的，未被导出。

PublicLib模块中的公有类

```
MyPublicLib.java x
1  package com.jinxuliang.libs;
2
3  import java.util.Random;
4
5  public class MyPublicLib {
6      //定义一个私有的字段
7      private static int secret = new Random().nextInt( bound: 100);
8      static {
9          System.out.println("secret的初始值为: "+secret);
10     }
11     //一个私有的静态方法, 获取secret字段的值
12     private static int getSecret(){
13         return secret;
14     }
15
16 }
```

注意一下MyPublicLib类中定义了一个私有的静态字段，通常情况下，这个字段外界是无法访问的，但通过反射是可以的。

MainApp模块中试图访问类型的私有成员

//通过反射访问类型的私有成员

```
private static void visitPrivateMemberViaReflection()
    throws NoSuchFieldException,
    IllegalAccessException,
    NoSuchMethodException,
    InvocationTargetException {
    var clazz : Class<MyPublicLib> = MyPublicLib.class;
    //获取私有字段的引用
    Field valueField = clazz.getDeclaredField( name: "secret");
    valueField.setAccessible(true);
    valueField.set(valueField, 100); //给它一个新值
    //获取私有方法的引用
    Method method = clazz.getDeclaredMethod( name: "getSecret");
    method.setAccessible(true);
    //调用静态方法
    var returnValue : Object = method.invoke( obj: null);
    //输出方法调用返回值
    System.out.println(returnValue);
}
```

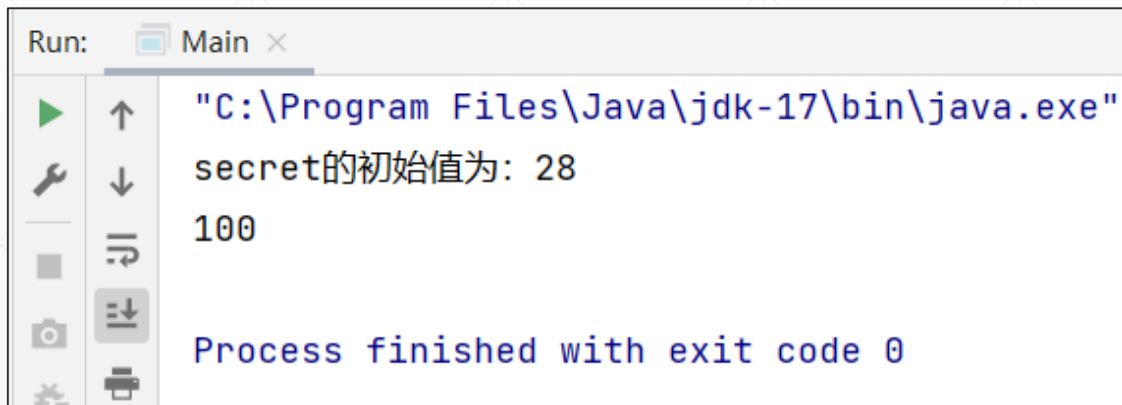
这个代码，将会引发异常：
InaccessibleObjectException

```
Exception in thread "main" java.lang.reflect.InaccessibleObjectException:
Unable to make field private static int
com.jinxuliang.libs.MyPublicLib.secret accessible: module PublicLib does
not "opens com.jinxuliang.libs" to module MainApp
    at
    java.base/java.lang.reflect.AccessibleObject.checkCanSetAccessible(Access
ibleObject.java:354)
    .....
```

开放包之后.....

```
module PublicLib {  
    exports com.jinxuliang.libs;  
    opens com.jinxuliang.libs;  
}
```

现在，反射代码运行正常：



```
Run: Main x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
secret的初始值为: 28  
100  
Process finished with exit code 0
```

通过在module-info.java中使用opens开放一个包，这个包中的代码，就支持外部代码通过反射技术读取。

这个例子说明，使用这种方式，可以存取类中的私有字段和调用类中的私有方法。下面，我们再来看看能不能访问非公有类.....

实例化模块内部非公有类-1

MyInternalClass 类归属于PublicLib模块，但它不是public的，并且它所在的包没有被模块导出：

```
MyInternalClass.java ×  
1 package com.jinxuliang.internal;  
2 //这个类及所在的包，都没有被模块所导出  
3 class MyInternalClass {  
4 }
```

实例化模块内部非公有类-2

```
//使用反射创建未导出的非公有类型实例
private static void createInstanceOfNonPublicClass()
    throws ClassNotFoundException,
    NoSuchMethodException,
    InvocationTargetException,
    InstantiationException,
    IllegalAccessException {
    //指定要实例化的类名
    String className="com.jinxuliang.internal.MyInternalClass";
    //加载类型
    var clazz : Class<?> = Class.forName(className);
    //获取构造函数引用
    var constructor : Constructor<?> =clazz.getDeclaredConstructor();
    //让其可被访问
    constructor.setAccessible(true);
    //创建实例
    Object target = constructor.newInstance();
    //证明实例确实已创建
    System.out.println(target);
}
```

MainApp模块中的代码，尝试着使用反射去实例化PublicLib模块中的MyInternalClass的实例。

实例化模块内部非公有类-2

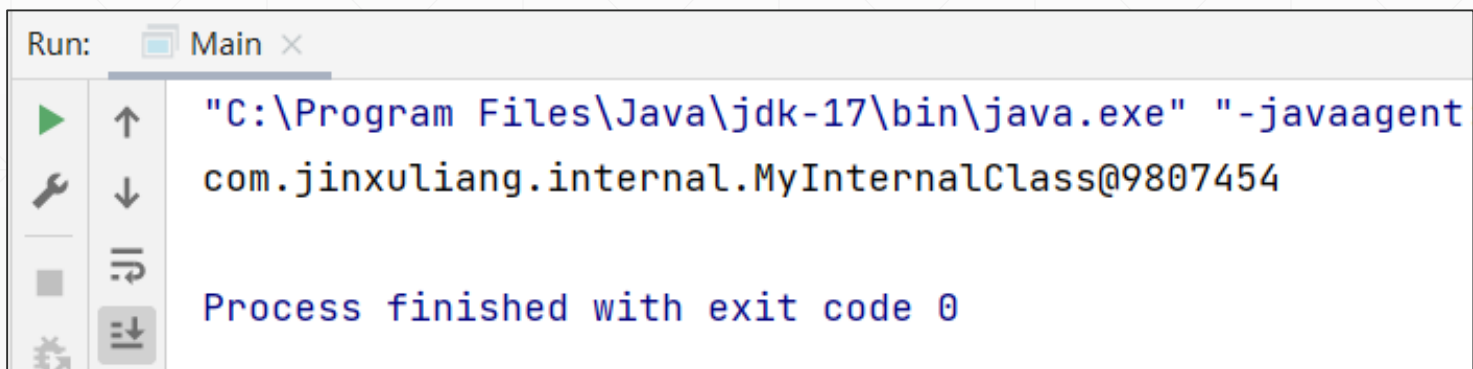
由于MyInternalClass对MainApp模块不可读，运行时会得到以下异常：

```
Exception in thread "main" java.lang.reflect.InaccessibleObjectException: Unable to make
com.jinxuliang.internal.MyInternalClass() accessible:
  module PublicLib does not "opens com.jinxuliang.internal" to module MainApp
    at java.base/java.lang.reflect.AccessibleObject.checkCanSetAccessible(AccessibleObject.java:354)
    at java.base/java.lang.reflect.AccessibleObject.checkCanSetAccessible(AccessibleObject.java:297)
    at java.base/java.lang.reflect.Constructor.checkCanSetAccessible(Constructor.java:188)
    at java.base/java.lang.reflect.Constructor.setAccessible(Constructor.java:181)
    at MainApp/com.jinxuliang.Main.createInstanceOfNonPublicClass(Main.java:38)
    at MainApp/com.jinxuliang.Main.main(Main.java:19)
```

限定开放

```
module PublicLib {  
    exports com.jinxuliang.libs;  
    opens com.jinxuliang.libs;  
    opens com.jinxuliang.internal to MainApp;  
}
```

现在，MyInternalClass 类可被实例化：

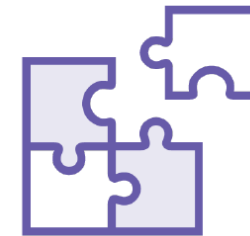


The screenshot shows a Java IDE's Run console. The title bar says "Run: Main x". The console output is as follows:

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent  
com.jinxuliang.internal.MyInternalClass@9807454  
  
Process finished with exit code 0
```

On the left side of the console, there are several icons: a green play button, a wrench, a square, and a bug icon. Next to these icons are arrows for navigating through the run history.

如果希望开放整个模块.....



只需在模块描述符中添加关键字 `open`，就可以开放一个模块：

```
open module mymodule{  
    ...  
}
```



开放的模块，它的所有包（不管是否导出），都支持“反射”，都是“开放式包（opened package）”。

区分运行时和编译时.....



在编译时，开放式包是强封装的，这时开放式包和非开放式包之间没有任何区别，遵循相同的限制和约定。这意味着，除了反射代码，其他那些尝试直接访问未导出的开放式包中类型的代码（比如创建类的实例），将无法通过编译。

简言之：**开放式包的作用不在编译期体现。**

在运行时，开放式包完全可以通过反射访问，包括非公有类、方法和字段，这就意味着现在使用反射，可以访问开放式包中的所有代码。



如果需要的话，可以通过在java命令行中使用`--add-opens`，动态地开放一个包。

模块化程序中的反射技术应用实例



JavaFX从JDK 10开始，就从JDK中移除，归第三方进行维护，而JavaFX已经彻底地模块化了，因此，在JavaFX模块化应用中，通常会添加以下声明，将应用程序所在的包，向javafx相关的模块开放，整个应用程序才能顺利运行：

```
module com.example.myjavafxmodules {  
    requires javafx.controls;  
    requires javafx.fxml;  
  
    opens com.example.testjavafxmodules to javafx.fxml;  
    exports com.example.myjavafxmodules;  
}
```



JavaFX依赖于反射读取控制器中的注解，所以，需要opens。